

LUVLOTS

Ownership Handover Guide

LUVLOTS — Ownership Handover Guide

This document is for whoever is receiving this app: a new developer, or the client taking over the project. It's written to be read **before** the main [README.md](#) — start here, then go to the README for deep technical detail (schema, RLS, payment adapter internals, testing checklist).

The short version: **this app currently runs entirely on the previous owner's personal accounts** (Supabase, Google Cloud, Ganap/G8 Pay, Expo/EAS). None of that transfers automatically. You need to create your own accounts for each service, then point the app at them. This guide walks through exactly that, in order.

1. What you're receiving

marketplace with auctions, Luvlist (wishlist), cart/checkout, and a seller dashboard.

three deployed payment Edge Functions (`supabase/functions/g8-pay-*`).

money, no sandbox (see README's "G8 Pay / Ganap specifics" section for the full list of what this integration does and doesn't cover — no refunds, no seller payouts, unsigned webhook).

- A React Native / Expo app (source code, this repo) — buyer/seller
- A live Supabase backend schema (migrations in `supabase/migrations/`) and
- A working Android build pipeline via EAS Build.
- **Auth**: email/password + Google Sign-In, both working end-to-end.
- **Payments**: G8 Pay (Ganap) QR Ph checkout for GCash/Maya/QR Ph — real

Read the README's "**Production readiness status**" section for the full, honest breakdown of what's real vs. "Coming Soon" (e.g. live selling, charity auctions, and direct messaging are intentionally unbuilt — no fake data was shipped for these).

2. Accounts you need to create (all yours, none inherited)

You cannot reuse the previous owner's logins for these — data/billing is tied to their personal accounts. Create your own for:

Service	Why	Free tier?
Supabase	Database, auth, file storage, payment Edge Functions	Yes
Google Cloud Console	Google Sign-In OAuth client	Yes
Expo / EAS	Building the Android APK/AAB, managing env vars	Yes (build minutes limited)
Ganap (G8 Pay)	Payment processing (GCash/Maya/QR Ph)	Ask Ganap for merchant onboarding — this is a live payments provider, not a sandbox

Create your own accounts rather than accepting access to the previous owner's — see Section 5 below for why. Follow the steps below on your own new accounts.

3. Setup order (do these in sequence)

Step 1 — Clone the repo and install dependencies

```
git clone https://github.com/Niiflheim01/LUVLOTS_FWDP.git
cd LUVLOTS_FWDP
npm install
```

Step 2 — Create your Supabase project and run migrations

1. Create a new project at supabase.com/dashboard.
2. Note the **Project URL** and **anon public key** (Settings -> API).
3. Run every file in `supabase/migrations/` in filename order, either via

the Supabase CLI:

```
npx supabase login
npx supabase link --project-ref <your-project-ref>
npx supabase db push
```

or by pasting each file into the Dashboard's SQL Editor in order.

4. This creates every table (`profiles`, `listings`, `orders`, etc.), RLS policies, and the three storage buckets (`listing-images`, `profile-images`, `verification-documents`).

Step 3 — Set up Google Sign-In on your own Google Cloud project

1. In [Google Cloud Console](#), create an OAuth 2.0 Client ID (type: **Web application**).
2. Add this Authorized redirect URI (use **your** Supabase project ref):

`https://<your-project-ref>.supabase.co/auth/v1/callback`

3. In your Supabase Dashboard -> **Authentication -> Providers -> Google**, paste the Client ID/Secret and enable the provider.

4. In your Supabase Dashboard -> **Authentication -> URL Configuration -> Redirect URLs**, add:

```
luvlots://auth/callback
luvlots://**
```

This step is easy to miss and causes a very specific, confusing bug: if it's not added, tapping "Continue with Google" completes the Google login, but the redirect back into the app fails and the in-app browser shows `localhost refused to connect` instead of returning to the app. That's Supabase silently falling back to its default (unconfigured) Site URL because it doesn't recognize the app's `luvlots://` redirect. If you ever see that error after changing Supabase projects, this is the first thing to check.

Step 4 — Local `.env` for development

```
cp .env.example .env
```

Fill in:

```
EXPO_PUBLIC_SUPABASE_URL=https://<your-project-ref>.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=<your-anon-key>
```

Run it locally to confirm auth/data works before touching payments or builds:

```
npm run dev
```

Step 5 — Ganap (G8 Pay) merchant account

1. Register a Ganap merchant project and get your **Project UUID** and **Secret Key**.

2. Set them as Supabase Edge Function secrets (never in `.env` — these are server-side only):

```
npx supabase secrets set G8PAY_MERCHANT_PROJECT_ID=<your-uuid> G8PAY_SECRET_KEY=<your-secret>
```

3. Deploy the three payment functions:

```
npx supabase functions deploy g8-pay-create-checkout
npx supabase functions deploy g8-pay-status
npx supabase functions deploy g8-pay-webhook --no-verify-jwt
```

4. In the Ganap dashboard -> your project -> Settings -> **Webhook URL**, paste:

```
https://<your-project-ref>.supabase.co/functions/v1/g8-pay-webhook
```

Ganap will refuse to create any checkout until this is filled in.

5. Read README's "**G8 Pay / Ganap specifics**" section before going live —

there is no sandbox, no refund path, and no confirmed seller payout support. Treat this as collecting real buyer payments only.

Step 6 — Expo/EAS account and building the app

1. Install the CLI and log in with **your own** Expo account:

```
npm install -g eas-cli
eas login
```

2. Re-link the project to your account (this changes the project ID):

```
eas init
```

This updates `extra.eas.projectId` in `app.json` — commit that change.

3. Register your env vars on EAS so cloud builds can see them (this is the

step that's easy to get wrong — see the warning box below):

```
eas env:create --scope project --name EXPO_PUBLIC_SUPABASE_URL --value "https://<your-project-ref>.supabase.co" --environment production
eas env:create --scope project --name EXPO_PUBLIC_SUPABASE_ANON_KEY --value "<your-anon-key>" --environment production
eas env:create --scope project --name EXPO_PUBLIC_APP_URL --value "luvlots://" --environment production
eas env:create --scope project --name EXPO_PUBLIC_ENABLE_G8_PAY --value "true" --environment production
```

4. Build:

```
eas build -p android --profile preview # installable .apk, for testing on a real device
eas build -p android --profile production # .aab, for Google Play submission
```

Why this matters — a real bug we hit during this handoff: `.env` is git-ignored on purpose (it's meant to stay local/secret), which means EAS Build in the cloud never sees it unless you register the same values with `eas env:create` as shown above. The app was built once *without* doing this and shipped with the values completely missing — the app showed "Supabase is not configured" on every screen. Registering the EAS env vars is not optional for cloud builds. There was a **second, subtler bug** on top of that: even after the EAS env vars were set correctly, the app still shipped without them, because `lib/env.ts` read them through an indirect variable (`const runtimeEnv = process.env;` `...runtimeEnv.EXPO_PUBLIC_X`) instead of the literal `process.env.EXPO_PUBLIC_X` form. Expo's build tooling only statically inlines the literal form — anything else silently evaluates to `undefined` in the compiled bundle, with no build error or warning. This has already been fixed in this repo (`lib/env.ts` now reads `process.env.EXPO_PUBLIC_X` directly), but if you ever add a *new* `EXPO_PUBLIC_*` variable, reference it the same direct way, or it will reproduce this exact failure mode.

4. Verifying a build actually works before distributing it

Because of the bug above, "the build succeeded" does not mean "the env vars are actually in the app." Before handing a build to anyone, extract and grep the bundle to confirm real values are present:

```
# after downloading the .apk from your build:
unzip -p your-build.apk assets/index.android.bundle > /tmp/bundle.js
grep -c "<your-project-ref>.supabase.co" /tmp/bundle.js # should be >= 1
grep -c "<first ~20 chars of your anon key>" /tmp/bundle.js # should be >= 1
```

If either comes back 0, the env vars didn't make it into the build — check the EAS env var registration (step 6.3 above) before re-building.

Then do a real device test, at minimum:

app instead of showing a browser error).

don't have to actually pay to confirm the flow renders).

- Sign up with email/password.
- Sign in with Google ("Continue with Google" — confirm it returns to the
- Add an item to cart and reach the GCash/Maya QR checkout screen (you

See the README's "**Testing checklist**" section for the full list, including RLS/security checks worth running with two separate accounts.

5. What to ask the previous owner for

Recommended: create fully independent accounts, don't inherit the previous owner's. Section 3 above already walks through creating your own Supabase project, Google Cloud project, and Expo/EAS account from scratch — that's the default path, not a fallback. Being added as a member/collaborator on someone else's personal Supabase organization or Google Cloud project means the app is still ultimately sitting on *their* account, under *their* billing, revocable at *their* discretion. For a real ownership handoff, don't do that — stand up your own accounts (Step 2, 3, and 6 in Section 3) and only ask the previous owner for what's needed to move data across, not for standing access:

own project (Section 3, Step 2), run the migrations yourself, and — only if there's real production data worth keeping — ask for a data-only export (`supabase db dump --data-only`) and storage bucket contents to import into your own project. Schema alone (the migrations in this repo) doesn't need anything from them at all.

Create your own Google Cloud project and OAuth Client ID (Section 3, Step 3) — this takes a few minutes and costs nothing. The only reason to ask the previous owner for anything here is if you want to preserve existing users' Google sign-in without forcing a re-consent, which usually isn't worth the ongoing dependency on their account.

(Section 3, Step 6) rather than accepting an Expo project transfer, so builds and app-store submissions are never tied to someone else's login.

merchant onboarding has to go through Ganap directly, and the current merchant account may be tied to Luvlots' business entity rather than any one person. **Ask David or Ms. Julia (CEO of Luvlots)** to either provide the existing Ganap merchant credentials for this business, or sponsor/introduce a new merchant application under Luvlots' name.

- **Supabase:** don't accept an invite to their organization. Create your
- **Google Cloud (OAuth):** don't accept IAM membership on their project.
- **Expo/EAS:** same logic — run `eas init` under your own Expo account
- **Ganap (G8 Pay) merchant account:** this one **can't be self-served** —

Once you're on your own accounts, ask the previous owner to **revoke whatever access they currently hold** (remove themselves as a Supabase project member, delete their local `.env/service` role key, etc.) — don't leave dual access lying around after a handoff.

6. Repository access — clone it, do not push to it

This repository (github.com/Niiflheim01/LUVLOTS_FWDP) stays under the previous owner's personal GitHub account.

Do not push commits, branches, or tags to this repository. Clone it to get the code, then move to your own repository for all future work — see the two options below.

1. **Clone it** to get the code:

```
git clone https://github.com/Niiflheim01/LUVLOTS_FWDP.git
```

2. Create your **own** repository (on your own GitHub account or your

organization's), then repoint your local clone's remote at it instead of pushing back to the original:

```
git remote remove origin
git remote add origin <your-new-repo-url>
git push -u origin final-handover # or whichever branch you're continuing from
```

Alternatively, use GitHub's **Fork** button on the original repo — that creates your own copy automatically without needing collaborator access at all.

If you were invited as a collaborator on the original repo for the handoff, that access should be **read-only**, and should be revoked once you've cloned and moved to your own copy — it's meant to get the code to you, not to be an ongoing shared repo.

7. Where to go next

the G8 Pay adapter's known limitations, project structure, and the complete testing checklist.

confirmed request/response shapes and documented gaps (no sandbox, no webhook signature, no refund API).

standalone-build redirect distinction explained inline.

- [README.md](#) — full technical reference: schema, RLS policies,
- `supabase/functions/_shared/g8pay-adapter.ts` — the payment integration's
- `lib/auth-context.tsx` — the auth flow, including the Expo Go vs.